



GraphRAG-R1: Graph Retrieval-Augmented Generation with Process-Constrained Reinforcement Learning

Chuanyue Yu
Nankai University
Binhai New Area, Tianjin, China
yuchuan Yue@mail.nankai.edu.cn

Kuo Zhao
Nankai University
Binhai New Area, Tianjin, China
zhaokuo@mail.nankai.edu.cn

Yuhan Li
HKUST Guangzhou
Guangzhou, Guangdong, China
yuhuanli98@gmail.com

Heng Chang*
Huawei Technologies Ltd
Haidian District, Beijing, China
changheng@gmail.com

Mingjian Feng
Nankai University
Binhai New Area, Tianjin, China
fengmingjian@mail.nankai.edu.cn

Xiangzhe Jiang
Nankai University
Binhai New Area, Tianjin, China
jiangxiangzhe@mail.nankai.edu.cn

Yufei Sun
Nankai University
Binhai New Area, Tianjin, China
yufei_sun@sina.com

Jia Li
HKUST Guangzhou
Guangzhou, Guangdong, China
jialeel@hkust-gz.edu.cn

Yuzhi Zhang*
Nankai University
Binhai New Area, Tianjin, China
zyz@nankai.edu.cn

Qingyun Sun
Beihang University
Haidian District, Beijing, China
sunqy@buaa.edu.cn

Jianxin Li
Beihang University
Haidian District, Beijing, China
lijx@buaa.edu.cn

Ziwei Zhang*[†]
Beihang University
Haidian District, Beijing, China
zwzhang@buaa.edu.cn

Abstract

Graph Retrieval-Augmented Generation (GraphRAG) has shown great effectiveness in enhancing the reasoning abilities of Large Language Models (LLMs) by leveraging graph structures for knowledge representation and modeling complex real-world relationships. However, existing GraphRAG methods still face significant bottlenecks when handling complex problems that require multi-hop reasoning, as their query and retrieval phases are largely based on pre-defined heuristics and do not fully utilize the reasoning potentials of LLMs. To address this problem, we propose GraphRAG-R1, an adaptive GraphRAG framework by training LLMs with process-constrained outcome-based reinforcement learning (RL) to enhance the multi-hop reasoning ability. Our method can decompose complex problems, autonomously invoke retrieval tools to acquire necessary information, and perform effective reasoning. Specifically, we utilize a modified version of Group Relative Policy Optimization (GRPO) that supports rollout-with-thinking capability to train the model. Next, we design two process-constrained reward functions. To handle the shallow retrieval problem, we design a Progressive Retrieval Attenuation (PRA) reward to encourage essential retrievals. Then, to handle the over-thinking problem, we design a Cost-Aware

F1 (CAF) reward to balance the model performance with computational costs. We further design a phase-dependent training strategy, containing three training stages corresponding to cold start and these two rewards. These stages empower GraphRAG with format following, behavior shaping, and smartness optimization abilities, respectively. Lastly, our method adopts a hybrid graph-textual retrieval to improve the reasoning capacity. Extensive experimental results demonstrate that GraphRAG-R1 significantly boosts LLM capabilities in solving complex reasoning problems compared to state-of-the-art GraphRAG methods on both in-domain and out-of-domain datasets. Furthermore, our framework can be flexibly integrated with various existing retrieval methods, consistently delivering performance improvements.

CCS Concepts

• **Computing methodologies** → **Knowledge representation and reasoning**; **Machine learning**.

Keywords

Reinforcement Learning, GraphRAG, Knowledge Graph

ACM Reference Format:

Chuanyue Yu, Kuo Zhao, Yuhan Li, Heng Chang, Mingjian Feng, Xiangzhe Jiang, Yufei Sun, Jia Li, Yuzhi Zhang, Qingyun Sun, Jianxin Li, and Ziwei Zhang. 2026. GraphRAG-R1: Graph Retrieval-Augmented Generation with Process-Constrained Reinforcement Learning. In *Proceedings of the ACM Web Conference 2026 (WWW '26)*, April 13–17, 2026, Dubai, United Arab Emirates. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3774904.3792589>

Resource Availability: The source code has been made publicly available at <https://doi.org/10.5281/zenodo.18349146> and can also be accessed

*Project co-leader.

[†]Corresponding author.



This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License.

WWW '26, Dubai, United Arab Emirates

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2307-0/2026/04

<https://doi.org/10.1145/3774904.3792589>

via the project repository: <https://github.com/ycygit/GraphRAG-R1>. The model weights are available at <https://doi.org/10.57967/hf/7629> and <https://huggingface.co/yuchuan Yue/GraphRAG-R1>.

1 Introduction

Large Language Models (LLMs) have attracted ever-growing attention in the past few years due to their outstanding ability in answering complex questions and generating human-quality text [40, 43]. Despite remarkable progress, LLMs still exhibit significant limitations when faced with complex tasks that require integrating vast and diverse knowledge domains or require deep reasoning and multi-step logical inference [30, 36]. As shown in Figure 1(a), LLMs tend to provide an incorrect response when the question requires knowledge beyond their scope. Graph Retrieval-Augmented Generation (GraphRAG) [13, 24, 29, 45, 53, 54], which utilizes knowledge graphs for knowledge representation, enables effective modeling of complex real-world relationships, such as causality, hierarchy, and dependency [5, 29].

However, existing GraphRAG methods still face significant performance bottlenecks when handling complex problems that require multi-hop reasoning. The main bottleneck is that existing methods rely on pre-defined or heuristic rules during query processing and retrieval, which are too rigid and inflexible. Concretely, during query processing, the semantic or lexical similarity search [6, 8, 13] remains the most widely adopted. For semantic similarity search, queries with high complexity are averaged out during vectorization, making it difficult to gather accurate information for each distinct semantic element. On the other hand, lexical similarity search captures only the superficial meaning of the query, easily losing implicit relationships and reasoning logic. Therefore, as shown in Figure 1(b), existing GraphRAG methods are unable to answer the query, since the retrieval results are incomplete or incorrect when queries are overly complex. Besides, during the graph retrieval phase, current methods employ a combination of graph traversal and subgraph retrieval for path reasoning, which are rather rigid [11, 20, 23, 25, 35]. For instance, ToG [25, 35], a representative GraphRAG method, starts from a node in the graph and traverses the surrounding nodes based on breadth-first search strategy with pruning. While this strategy might work for short paths, it struggles with long-path problems, which require capturing logically sound but distant or even disconnected relationships. This inflexibility often leads to broken reasoning chains and failed retrievals, making it unsuitable for multi-hop problems with complex reasoning logic. As a result, the existing GraphRAG methods fail to fully leverage the LLM's deep reasoning potential.

In this paper, we try to further enhance the multi-hop reasoning ability of GraphRAG. Reinforcement learning (RL) has recently been shown to be effective in endowing LLMs with powerful reasoning abilities. For example, in the case of DeepSeek-R1 [10], it has been discovered that large-scale reinforcement learning can endow models with deep thinking capabilities [18, 37, 46], even without any supervised fine-tuning. This effectiveness may result from RL's use of reward signals to encourage exploration, refining dynamic reasoning strategies from static knowledge [27] beyond static reasoning patterns [1, 9, 47, 52]. However, how to utilize RL for GraphRAG training remains unexplored in the literature.

The major technical obstacle lies in how to design proper reward functions so that the reasoning process of LLMs within GraphRAG can be properly constrained, avoiding potential problems such as shallow retrieval [4, 15] and over-thinking [2, 3, 34].

To address this issue, we propose GraphRAG-R1, an adaptive GraphRAG framework with deep reasoning capabilities through process-constrained outcome-based RL. As shown in Figure 1(c), our method can decompose complex multi-hop problems by reasoning and autonomous invocation of retrieval tools. Specifically, we employ a modified version of Group Relative Policy Optimization (GRPO) that supports rollout-with-thinking mechanism [33] to leverage external retrieval tools on demand. Next, we design two process-constrained reward functions, Progressive Retrieval Attenuation (PRA) and Cost-Aware F1 (CAF), aiming to inject prior knowledge into the LLM's training process. In detail, PRA aims to dynamically adjust the reward for retrieval actions. It encourages necessary retrievals, preventing the model from shallow retrieval. As the reasoning deepens, PRA progressively attenuates retrieval rewards, guiding the model to avoid unnecessary retrieval overhead. On the other hand, CAF evaluates the final answer correctness using the F1 score while also accounting for the computational resources consumed during the reasoning and retrieval process. It encourages the model to achieve correct results more efficiently to prevent over-thinking. These two rewards collectively balance effectiveness and computational cost. We further design a three-stage phase-dependent training strategy. In the first stage, we focus on the answer format to alleviate cold start problems. In the second and third stages, we focus on improving the answer quality using the aforementioned two rewards. Lastly, our method employs a hybrid graph-textual retrieval. By leveraging graph structures to precisely capture entity relationships while preserving the rich semantic details of the original text, our method provides more comprehensive and integrated knowledge for complex reasoning.

Experiments show that GraphRAG-R1 significantly outperforms existing GraphRAG methods in solving complex reasoning problems, across both in-domain and out-of-domain datasets. Additionally, our framework offers flexible integration with various retrieval methods, consistently enhancing their performance.

Our main contributions are summarized as follows:

- We propose GraphRAG-R1, a pioneering framework that utilizes GRPO that supports rollout-with-thinking to enhance the reasoning ability of GraphRAG.
- We propose PRA and CAF, two tailored reward designs to guide the training process of RL and handle the shallow retrieval and over-thinking problems. We also propose a phase-dependent training strategy and a hybrid graph-textual retrieval.
- Experiments demonstrate that our method exhibits state-of-the-art performance on both in-domain and out-of-domain datasets, and can be flexibly integrated with various existing retrieval methods with consistently improved performance.

2 Related Work

2.1 Graph Retrieval-Augmented Generation

Graph Retrieval-Augmented Generation (GraphRAG) offers a novel paradigm to enhance the reasoning abilities of LLMs by explicitly modeling entity relationships through graph structure such as

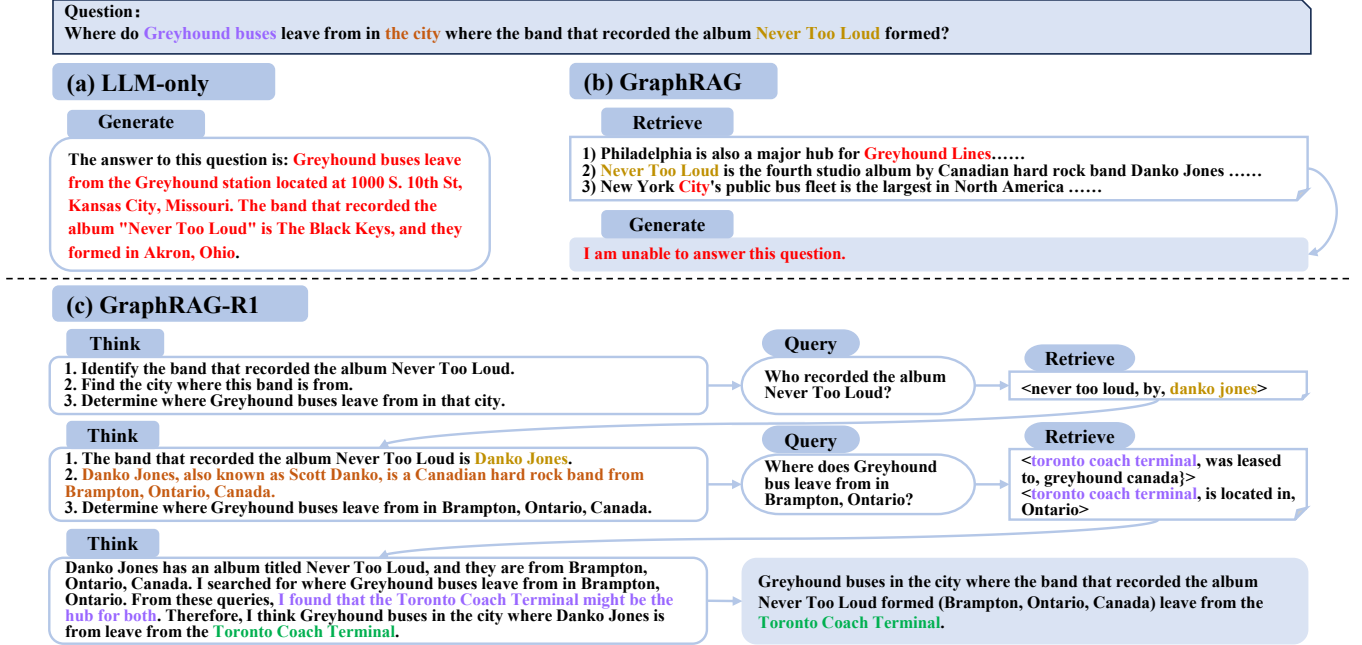


Figure 1: An example of comparing using only LLMs, GraphRAG, and GraphRAG-R1 in answering complex problems: (a) The LLM (Qwen2.5-7B) directly produces an output, but the answer is incorrect, (b) HippoRAG2 [12] enhances the LLM (Qwen2.5-7B) by retrieving external knowledge, but fail to generate correct outputs for complex problem, (c) our method successfully decomposes the problem and autonomously invokes retrieval methods, producing the correct output.

knowledge graphs [13, 29]. GraphRAG has been widely adopted across multiple domains. For instance, GraphRAG based on knowledge graph [28] incorporates techniques like subgraph retrieval and Graph Neural Networks (GNNs) [14] to enhance the accuracy and explainability of LLMs in complex knowledge reasoning question answering. GraphRAG based on document graphs [32] models semantic relationships between documents and internal document structures, effectively supporting long-document summarization and cross-document information integration [31]. Furthermore, GraphRAG methods for specialized domains are increasingly trending. Examples include leveraging social graphs [19, 51] to analyze user relationships for precision recommendations and constructing medical or disease graphs [22, 48] that integrate multi-source heterogeneous data to enable personalized treatment plan generation.

However, existing GraphRAG methods still face significant performance bottlenecks when facing complex reasoning problems. For example, ToG [25, 35] introduces reasoning steps using breadth-first search (BFS) with pruning approach, but it heavily relies on external LLMs and thus lacks flexibility. G-Retriever [14] adopts K-Nearest Neighbors-based retrieval with limited efficacy in multi-hop scenarios. Moreover, existing approaches suffer from shallow retrieval depth, since they often depend on predefined fixed paths or hop counts. HippoRAG [12, 20] only supports single-hop retrieval. KGP [44] and LighRAG [11] are typically confined to 2-3 hops, exhibiting performance degradation when handling problems that require deep-dependence mining. In contrast, our work improves the reasoning ability of GraphRAG for complex problems using process-constrained RL, aiming to effectively resolves the issues of rigid retrieval strategies and complex reasoning shortcomings in existing GraphRAG approaches.

2.2 RL-Enhanced Retrieval Scheme

Recently, reinforcement learning (RL) has become increasingly important for complex reasoning [27, 46, 49]. For example, DeepSeek-R1 [10] shows that large-scale RL can endow LLMs with deep thinking capabilities. Some pioneering works have also adopted RL to enhance RAG [21, 33]. For example, R1-Searcher [33] and Search-R1 [21] achieve co-training of retrieval and generation, effectively improving the model’s ability to solve complex problems. However, a systematic exploration of RL schemes within the GraphRAG domain remains open. Technically speaking, directly extending existing RL methods into GraphRAG faces several challenges. Firstly, the existing methods usually employ a single outcome-oriented reward function, e.g., based solely on the correctness of final answer. However, this design becomes too simple for GraphRAG, since a complex problem may involve complex retrieval calls of the external knowledge graph. More critically, existing methods have often encountered reward hacking in RL training, potentially resulting in shallow retrieval and over-thinking in GraphRAG. To tackle these issues, we propose GraphRAG-R1 with tailored reward functions and training strategy to non-trivially explore the potential of RL in enhancing GraphRAG.

3 Method

In this section, we present our proposed method. First, we introduce the rollout retrieval enhanced GRPO algorithm. Second, we detail the designs of process-constrained reward, which includes PRA and CAF, together with the phase-dependent training strategy. Finally, we present our hybrid graph-textual retrieval design. An overall framework of our method is shown in Figure 2.

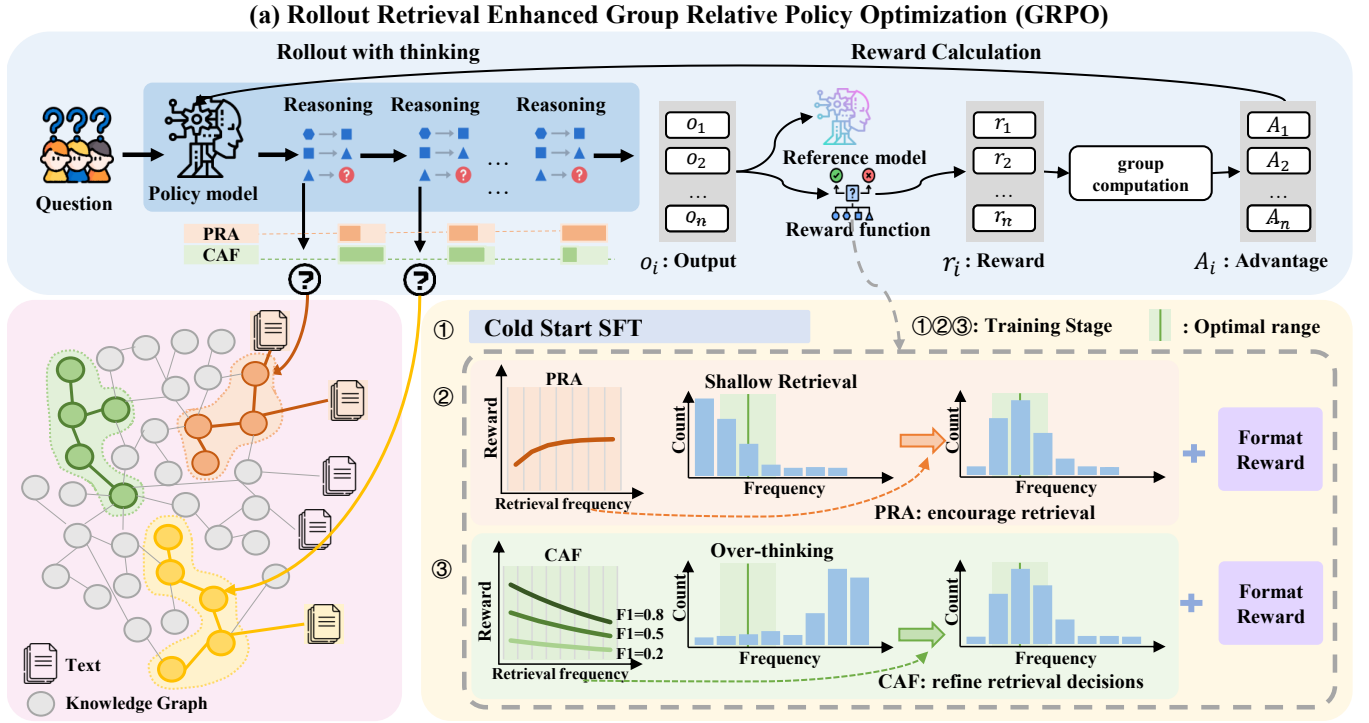


Figure 2: An overview of GraphRAG-R1 : (a) the rollout retrieval enhanced GRPO as the training strategy of LLM, (b) process-constrained reward designs, containing PRA and CAF rewards and the phase-dependent training strategy, (c) the hybrid graph-textual retrieval, which is more informative than text fragments.

3.1 Rollout Retrieval Enhanced GRPO

In this subsection, we introduce the details of our training method, which is a modified version of GRPO that supports the roll-out-with-thinking capability [33].

Our goal is to optimize the LLM using RL to generate high-quality outputs augmented by an external retrieval tool $\mathcal{R}$ that assists the reasoning generation process. Specifically, denote the policy of RL as $\pi_\theta(\cdot)$ parametrized by θ . Let q represent input queries drawn from the training dataset $\mathcal{D}$, o represent the generated outputs, and $r(\cdot)$ be the reward function. We formalize the objective as follows:

$$\max_{\pi_\theta} \mathbb{E}_{q \sim \mathcal{D}, o \sim \pi_\theta(\cdot|q; \mathcal{R})} [r(q, o) - \beta \cdot \mathbb{D}_{\text{KL}}(\pi_\theta \| \pi_{\text{ref}})], \quad (1)$$

where $\pi_{\text{ref}}(\cdot)$ is the reference LLM, $\mathbb{D}_{\text{KL}}(\cdot)$ is the KL-divergence measure, and β is a hyperparameter that controls the strength of the KL-divergence regularization. More specifically, our method builds on the GRPO algorithm [10], which uses the average reward of multiple sampled outputs as a baseline. During training, for each input query q , GRPO samples a group of G responses $o_1, o_2, \dots, o_G$ using the old policy $\pi_{\theta_{\text{old}}}$. The policy model is then optimized by maximizing the following objective function:

$$\mathcal{J}_{\text{GRPO}}(\theta) = \mathbb{E}_{\substack{q \sim \mathcal{D} \\ \{o_i\}_{i=1}^G \sim \pi_{\theta_{\text{old}}}}} \left[\frac{1}{G} \sum_{i=1}^G \min \left(\frac{\pi_\theta(o_i | q)}{\pi_{\theta_{\text{old}}}(o_i | q)} \cdot A_i, \right. \right. \\ \left. \left. \text{clip} \left(\frac{\pi_\theta(o_i | q)}{\pi_{\theta_{\text{old}}}(o_i | q)}, 1 - \epsilon, 1 + \epsilon \right) \cdot A_i \right) - \beta \cdot \mathbb{D}_{\text{KL}}(\pi_\theta \| \pi_{\text{ref}}) \right], \quad (2)$$

where the group size G determines the number of responses generated per query, θ_{old} denotes the previous policy parameters, ϵ is the clipping range hyperparameter to constrain policy updates, and A_i represents the advantage, computed based on the relative rewards of outputs within each group as follows:

$$A_i = \frac{r_i - \mu_r}{\sigma_r}, \quad (3)$$

where r_i is the reward for individual output, μ_r means the average reward of all G outputs for the same query q , and σ_r is the group reward standard deviation.

To better adapt GRPO to our GraphRAG scenario, we adopt two key modifications proposed by R1-Searcher [33]: Rollout-with-Thinking and Retrieval-Masked Loss. The core objective is to enable the model to autonomously leverage external retrieval tools on demand during inference and receive rewards for generating correct answers.

3.1.1 Rollout-with-Thinking. The model is guided to utilize the external retrieval tool during generation by using two special tokens $\langle \text{begin_of_query} \rangle \dots \langle \text{end_of_query} \rangle$ to indicate the call to the search tool. When $\langle \text{end_of_query} \rangle$ appears, it signifies that the retrieval query has been fully generated. At this point, model generation is paused, the external retrieval tool is called, and the retrieved content is marked with $\langle \text{begin_of_documents} \rangle \dots \langle \text{end_of_documents} \rangle$. This content then guides the model's next step of reasoning. This method ensures retrieval is seamlessly integrated into the reasoning process.

3.1.2 Retrieval-Masked Loss. In the original GRPO, token-level loss is computed throughout the rollout sequence. In the GraphRAG scenario, the rollout sequence consists of tokens generated by the LLM and tokens retrieved from external channels. Applying the same optimization to retrieved tokens and LLM-generated tokens may negatively impact the training process. Therefore, we utilize special tokens `<|begin_of_documents|>` and `<|end_of_documents|>` and their enclosed content is masked during training, ensuring the policy gradient objective is calculated only on LLM-generated tokens.

3.2 Process-Constrained Reward Design

In this subsection, we introduce our reward functions to constrain the retrieval process, enabling effective and efficient reasoning.

RL methods have recently attracted considerable attention to improve the reasoning abilities of LLMs. However, if simple reward functions are employed, such as relying solely on final answer correctness, existing methods easily face a technical problem known as “Reward Hacking” [7], i.e., the model exploits certain loopholes in the reward to gain high scores rather than genuinely improving reasoning capabilities. To address this potential issue, our reward design injects controlled human prior knowledge while preserving the fundamental reward framework. Concretely, we introduce the following three rewards: format reward, PRA, and CAF, as follows.

3.2.1 Format reward. This reward function is designed to steer the model toward strictly adhering to the specified output format, ensuring that it learns to correctly invoke external retrieval and generate the final answer. The model should place the final response entirely within the `<answer>...</answer>` tags and ensure that the entire output is fully readable and free of uninterpretable artifacts. When invoking retrieval, the model should encapsulate retrieval queries within the `<|begin_of_query|>...<|end_of_query|>` tags with retrieved information contained within `<|begin_of_documents|>...<|end_of_documents|>`. The model should explicitly invoke the retrieval mechanism through this query process rather than directly generating content without retrieval.

The format reward is defined as follows:

$$R_{\text{format}} = \begin{cases} 0.5, & \text{correct} \\ 0, & \text{incorrect.} \end{cases} \quad (4)$$

3.2.2 Progressive Retrieval Attenuation (PRA). The most straightforward design to encourage GraphRAG models to invoke retrieval calls is to adopt a single retrieval reward. However, this strategy tends to cause the model to invoke the external retrieval only once and cause the *shallow retrieve* problem, as some complex multi-hop problems require multiple retrieval calls. On the other hand, one can sum up rewards for multiple retrievals, but it may trigger unnecessary retrieval calls. To circumvent these issues, we propose Progressive Retrieval Attenuation (PRA), specifically designed to provide tailored rewards for each invocation while incorporating a decay factor. With each additional retrieval call, the reward undergoes exponential decay relative to the base reward, concurrently accumulating rewards from the preceding invocations. Formally, the retrieval reward is defined as follows:

$$R_n = \begin{cases} R_0, & n = 1 \\ R_{n-1} + R_0 \times k^{n-1}, & n > 1 \end{cases}, \quad (5)$$

where R_0 is the base reward, n is the number of retrieval calls, and $0 < k < 1$ is the decay factor. Through this dynamically adjusted reward weighting, the retrieval frequency of the model can be precisely controlled within the desired threshold ranges. Using the properties of geometric series, it is easy to see that:

$$R_0 \leq R_n < \frac{1}{1-k} R_0, \quad \forall n. \quad (6)$$

When $k = 0$, PRA degenerates into a single retrieval reward. When $k = 1$, it degenerates into the sum of rewards. The final reward is $R_{\text{retrieval}} = R_N$, where N is the actual number of retrieval calls.

Through this reward function, we can effectively restrict the number of retrieval calls, effectively preventing shallow retrieval. For different data characteristics, the hyperparameter k can be flexibly adjusted to better steer the model’s learning process.

3.2.3 Cost-Aware F1 (CAF). Although the PRA reward can regulate the retrieval frequency of the model within a reasonable range, this mechanism alone still relies on prior knowledge and could not fully activate the reasoning capabilities of LLMs. For example, the LLM may adopt unnecessarily numerous retrievals, leading to the “over-thinking” problem and greatly waste the computational resources. An ideal GraphRAG model should deliver high-quality responses with precisely needed retrieval operations, which requires three core competencies: accurately assessing the problem difficulty, intelligently decomposing complex queries, and dynamically optimizing retrieval strategies.

To this end, we design the CAF reward, which achieves the above objective through another decay coefficient. Formally, the CAF reward is defined as:

$$R_{\text{CAF}} = F1 \times a \times e^{-b \times N}, \quad (7)$$

where $F1$ is the F1 score of the generated answers, a and b are hyperparameters to control the coefficients in the regulatory function, and N denotes the total number of retrieval operations. The CAF reward ensures that responses achieving identical F1 score receive higher rewards when using fewer retrievals, thereby incentivizing the model to proactively recognize problem complexity and refine retrieval decisions. Consequently, CAF reward establishes a balance among reasoning depth, retrieval cost, and generation quality, effectively mitigating suboptimal behaviors of over-thinking.

3.2.4 Phase-Dependent Training Strategy. Despite the effectiveness of these reward functions, simultaneously optimizing them induces technical challenges. To address this challenge, we propose a phase-dependent training strategy to avoid the potential conflicts between objectives. Concretely, we split the training into three sequential stages, detailed as follows.

Specifically, in the first stage, we utilize another LLM to generate data following our desired format. Then, we utilize supervised fine-tuning (SFT) to train the LLM with the generated data. This training process can enable the model to follow our desired format by addressing the cold start problem and stabilizing the training process, laying the foundation for the following training process.

In the second stage, we aim to shape the behavior of the model, synergistically combining the format reward and the PRA reward. This design recognizes standardized tool-calling protocols and retrieval behavior as essential prerequisites for high-quality reasoning. By learning with two objectives, the model can efficiently learn

how and when to invoke the retrieval tools with structured output patterns. The decay mechanism of PRA dynamically regulates retrieval behavior through a high initial reward, driving essential information retrieval, while its exponential decay characteristic suppresses redundant operations.

In the last stage, we further optimize the overall smartness of the model, adopting CAF as the primary reward objective while retaining format rewards to prevent forgetting issues. As discussed in Section 3.2.3, CAF establishes a trade-off between maximizing answer correctness and constraining retrieval counts. This incentivizes models to adaptively adjust retrieval depth based on query complexity, suppressing non-essential retrievals for simple queries while expanding information acquisition for complex multi-hop problems, leading to effective and efficient reasoning.

In summary, our phase-dependent training strategy resolves potential gradient conflicts, improving training stability and significantly boosting model performance.

3.3 Hybrid Graph-Textual Retrieval

Compared to conventional RAG, GraphRAG exhibits superior relational modeling capabilities using graph structure. We integrate this advantage into both the training and generation phases of our framework. Specifically, we retrieve both structured triplets and relevant texts, replacing text fragments as commonly retrieved in existing RAGs and GraphRAGs with hybrid graph-textual retrieval.

By employing hybrid retrieval, our method has several advantages. Compared to lengthy text fragments, the compact nature of graph can drastically reduce token consumption, improving processing efficiency. This structured representation allows the model to focus directly on essential entity-relationships while inherently filtering out the redundant noise. Therefore, the integration of structured knowledge significantly enhances the quality and efficiency of generated answers, as reasoning operates on a cleaner and more relevant information foundation. We empirically verify the hybrid graph-textual retrieval in Section 4.3.3.

4 Experiments

In this section, we conduct experiments to evaluate GraphRAG-R1, in order to answer the following research questions:

- **RQ1:** Does GraphRAG-R1 effectively improve the performance of existing LLMs for both in-domain and out-of-domain datasets?
- **RQ2:** Do all components, including PRA and CAF rewards, and phase-dependent training, contribute positively to our method?
- **RQ3:** Is GraphRAG-R1 flexible enough to be seamlessly integrated with other retrievers?

4.1 Experimental Setup

4.1.1 Dataset. To comprehensively evaluate the GraphRAG-R1 framework, we follow [20, 41] and select four widely used multi-hop QA benchmark datasets: HotpotQA [50], 2Wiki [16], MuSiQue [39], and PopQA [26]. For the training process, we utilize 80% of the training data from HotpotQA, 2Wiki, and MuSiQue. The remaining 20% of these datasets are employed as the in-domain testing set. Meanwhile, we utilize PopQA as an out-of-domain testing set, aiming to compare the generalization capabilities of different models. More details are provided in Appendix A.6.

4.1.2 Metrics. Following the common literature, we employ F1 Score [44], SBERT Similarity (SBERT) [38], and LLM-as-Judge Accuracy (ACC_L) [33] as evaluation metrics. The F1 Score measures the lexical overlap between generated answers and ground-truth references. SBERT uses the cosine similarity between sentence embeddings generated by the SBERT model to measure semantic similarity. ACC_L leverages an LLM (specifically, Qwen3-Turbo) as an evaluator.

4.1.3 Baselines. We compare 7 representative methods:

- **KGP** [44]: It utilizes fine-tuned LLMs to explore neighboring nodes and aggregate information from a KG to produce answers.
- **ToG** [35]: It dynamically expands and evaluates the most probable inference path in the KG to perform reasoning.
- **LightRAG** [11]: It enhances standard RAG by integrating graph-based text indexing with a dual-level retrieval strategy.
- **PropRAG** [41]: It optimizes the reasoning chain via propositional representation and beam search.
- **G-Retriever** [14]: It integrates LLM and graph neural network (GNN) with RAG for conversational graph QA.
- **HippoRAG2** [12]: It uses personalized PageRank for deep passage integration inspired by the hippocampal indexing theory.
- **R1-Searcher** [33]: It enhances the capability of LLMs to invoke RAG for retrieval through a two-stage RL framework.

We also compare with the following self-constructed baselines:

- **Vanilla LLM:** It uses the raw backbone LLM for inference.
- **Naïve RAG:** It adopts a single-hop retrieval from the vector database for generation.
- **Prompt Engineering:** It adopts GraphRAG via structural prompts.
- **SFT:** It adopts GraphRAG via SFT on the KGQA data.

4.1.4 Implementation Details. We utilize Qwen2.5-7B as the default backbone LLM model. For baselines that require external closed-source LLMs, we use Qwen3-Turbo. We use HippoRAG2 as the external retriever for LLM’s rollout retrieval.

For training, in the first stage, we generate training data using ERNIE-3.5-128K [42]. In the second stage, we set the base reward for PRA as $R_0 = 0.5$ and its decay factor as $k = 0.5$. In the third stage, we set the hyper-parameter for CAF as $a = 2$ and $b = 0.1$. Throughout three stages, LoRA fine-tuning [17] is employed with $r = 16$ and $\text{lora_alpha} = 32$. The sampling temperature is set to 1.0 with a maximum retrieval count of 8, the learning rate is $5e - 6$, and the number of epochs for SFT is 1.

4.2 Main Results

To answer **Q1**, we present the performance comparison between GraphRAG-R1 and baselines across four benchmarks in Table 1. We can draw the following three observations:

GraphRAG-R1 achieves consistent improvements across all datasets. Specifically, we achieve the best results for evaluation metrics F1 and SBERT in all datasets, and achieve three best results and one third-best result for ACC_L . This fully validates the effectiveness of the GraphRAG-R1 framework in enhancing LLMs’ capability to handle multi-hop reasoning problems. Particularly, it outperforms the strongest baseline in F1 score by up to 38.08% on HotpotQA, 62.43% on MuSiQue, and 83.81% on 2Wiki, respectively. GraphRAG-R1 gets more improvement on a more difficult multi-hop dataset.

Table 1: The results of different methods for multi-hop question answering. The first three datasets are in-domain, and PopQA is out-of-domain (i.e., unseen during training). The best results are in bold and the second-best results are underlined.

Method	HotpotQA			MuSiQue			2Wiki			PopQA		
	F1	ACC _L	SBERT	F1	ACC _L	SBERT	F1	ACC _L	SBERT	F1	ACC _L	SBERT
Vanilla LLM	3.08	30.00	44.32	2.01	15.50	43.14	4.55	29.50	44.37	10.46	25.90	50.07
Naive RAG	24.24	49.00	58.73	8.99	19.00	48.97	13.14	25.00	52.56	23.72	<u>58.10</u>	58.60
Prompt Engineering	22.22	41.00	58.67	7.72	<u>25.50</u>	49.91	<u>17.54</u>	36.00	<u>54.44</u>	13.37	27.50	51.61
SFT	12.10	49.50	51.76	8.29	21.00	48.90	11.40	<u>48.50</u>	49.46	10.45	54.10	50.56
KGP	10.73	21.00	50.92	4.61	11.00	46.70	10.16	18.00	50.27	21.01	50.00	56.10
ToG	11.44	21.50	50.48	5.02	8.00	47.05	14.45	21.50	53.11	<u>29.21</u>	47.00	<u>62.94</u>
LightRAG	7.96	18.50	49.15	3.09	6.00	44.99	7.58	10.00	47.95	14.55	34.30	53.07
PropRAG	25.47	55.00	59.76	10.11	21.50	<u>50.77</u>	17.17	35.00	53.85	23.29	57.30	58.22
G-retriever	14.81	21.50	54.89	3.38	5.00	47.47	15.12	19.00	52.77	2.14	6.80	44.63
HippoRAG2	<u>27.52</u>	<u>53.00</u>	<u>61.02</u>	8.90	23.50	50.19	12.32	29.00	51.92	23.99	57.46	58.16
R1-Searcher	26.82	35.00	59.15	<u>12.35</u>	14.50	50.32	15.89	22.50	51.51	23.24	43.20	56.80
GraphRAG-R1	38.00	52.00	66.66	20.06	26.50	57.49	32.24	49.50	63.56	35.04	60.00	65.31

Table 2: The ablation study results on PRA and CAF rewards. #Calls denotes the number of retrieval calls.

Method	HotpotQA		MuSiQue		2Wiki		PopQA	
	F1	#Calls	F1	#Calls	F1	#Calls	F1	#Calls
GraphRAG-R1	38.00	1.94	20.06	2.36	32.24	2.05	35.04	1.49
- PRA	<u>33.92</u>	1.77	14.48	2.22	<u>31.55</u>	1.75	<u>31.40</u>	1.32
- CAF	28.36	2.33	<u>19.82</u>	2.80	22.16	2.36	26.98	2.05
- PRA & CAF	33.34	1.83	15.32	2.26	28.94	1.94	30.06	1.37

GraphRAG-R1 achieves leading performance on both in-domain datasets and the out-of-domain dataset PopQA. Concretely, it outperforms the best baseline by up to 19.96% in terms of F1 on PopQA, indicating that our proposed method possesses strong generalization ability and can effectively handle unseen complex questions.

From the results of self-constructed baselines, existing techniques such as naive RAG, prompt engineering, and SFT can indeed improve the ability of vanilla LLM on complex question answering tasks. However, our method, which adopts RL-based training, is clearly more effective, usually outperforming all these non-RL approaches by a large margin. The consistent performance improvement validates that RL-based methods hold more potential in enhancing GraphRAG and the reasoning ability of LLMs in general.

4.3 Further Analyses

To answer Q2, in this section, we conduct further analyses to verify the designs of our reward functions, phase-dependent training strategy, and hybrid retrieval. We also discuss the general compatibility of our method with various retrieval methods to answer Q3.

4.3.1 PRA & CAF Rewards. We perform ablation studies to evaluate the contribution of PRA and CAF reward designs. We remove PRA, CAF, and both rewards, denoted as “-PRA”, “-CAF”, and “-PRA & CAF”, respectively, and present the results in Table 2. The results show that removing either PRA or CAF, or both, leads to substantial performance drops in terms of F1 score. This confirms that both mechanisms are indispensable components of GraphRAG-R1.

Besides the quantitative F1 score, we also delve deeper into the results by reporting the number of retrieval calls. We make the

Table 3: The results of the ablation study on the phase-dependent training strategy.

Method	HotpotQA		MuSiQue		2Wiki		PopQA	
	F1	ACC _F	F1	ACC _F	F1	ACC _F	F1	ACC _F
GraphRAG-R1	38.00	92.00	20.06	89.50	32.24	89.00	35.04	95.10
- cold start	<u>30.31</u>	<u>89.00</u>	14.84	<u>89.50</u>	<u>30.04</u>	<u>89.50</u>	28.83	<u>92.80</u>
- all stages	26.12	71.00	<u>16.79</u>	74.00	28.05	76.50	<u>30.23</u>	82.50

following observations. Without using PRA, our method has a reduced number of retrieval calls, indicating that it suffers from the “shallow retrieval” problem and leads to unsatisfactory performance. PRA makes the retrieval behavior more controllable.

On the other hand, without using CAF, the number of retrieval calls consistently increases across all datasets, indicating the model is more inclined to invoke external functions. However, this not yield to corresponding F1 improvements in answer quality. In fact, the F1 score is even lower than methods invoking few retrievals, indicating that the model tends towards “over-thinking” and performing unnecessary retrievals.

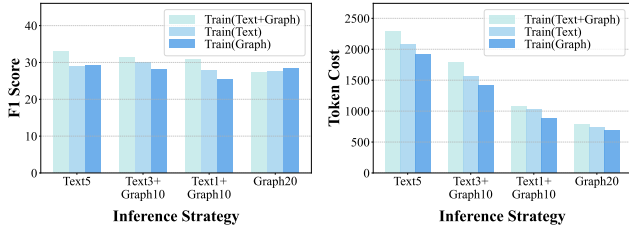
When both rewards are adopted, our method achieves the best performance with a suitable number of retrieval calls, indicating our method can calibrate its retrieval strategy to the inherent difficulty of the problem. For HotpotQA and PopQA, which contain relatively simpler problems, the number of retrieval calls is automatically decreased. Conversely, for inherently more complex problems demanding deeper information gathering, such as MuSiQue and 2Wiki, our model learns to invoke retrievals more frequently.

4.3.2 Phase-Dependent Training Strategy. To validate the effectiveness of our training strategy, we compare our method with two variants: one without the first cold-start stage (denoted as “-cold start”), and the other without distinct stages, i.e., directly mixing all reward functions for training (denoted as “-all stages”). The results are shown in Table 3. Here, ACC_F represents the percentage of model outputs that are entirely accurate in format.

Introducing the cold-start strategy effectively improves performance. In this stage, the model can learn simple tasks like format

Table 4: The results of integrating GraphRAG-R1 with other GraphRAG retrieval methods. Note that GraphRAG-R1 is only trained once and do not access other RAG methods during training. Imp denotes the average improvement across all datasets.

Method	Model	HotpotQA			MuSiQue			2Wiki			PopQA			Imp
		F1	ACC _L	SBERT	F1	ACC _L	SBERT	F1	ACC _L	SBERT	F1	ACC _L	SBERT	
Naive RAG	Original	24.24	49.00	58.73	8.99	19.00	48.97	13.14	25.00	52.56	23.72	58.10	58.60	+ 18.04%
	+GraphRAG-R1	32.08	48.00	62.72	20.77	28.00	56.59	30.14	37.50	61.22	31.96	57.50	52.96	
KGP	Original	10.73	21.00	50.92	4.61	11.00	46.70	10.16	18.00	50.27	21.01	50.00	56.10	+ 36.25%
	+GraphRAG-R1	36.16	41.50	65.81	13.33	14.50	53.82	27.79	35.50	60.41	18.33	55.00	55.39	
ToG	Original	11.44	21.50	50.48	5.02	8.00	47.05	14.45	21.50	53.11	29.21	47.00	62.94	+ 5.26%
	+GraphRAG-R1	16.82	25.50	55.36	8.29	8.50	51.77	21.95	26.00	57.68	22.30	37.00	60.10	
LightRAG	Original	7.96	18.50	49.15	3.09	6.00	44.99	7.58	10.00	47.95	14.55	34.30	53.07	+ 38.37%
	+GraphRAG-R1	20.85	27.00	57.40	8.59	12.50	48.95	19.64	23.00	55.31	28.42	49.00	60.50	
PropRAG	Original	25.47	55.00	59.76	10.11	21.50	50.77	17.17	35.00	53.85	23.29	57.30	58.22	+ 13.16%
	+GraphRAG-R1	30.87	42.00	62.25	18.78	21.00	55.32	34.86	45.50	64.44	32.69	57.80	63.44	
G-retriever	Original	14.81	21.50	54.89	3.38	5.00	47.47	15.12	19.00	52.77	2.14	6.80	44.63	+ 6.18%
	+GraphRAG-R1	13.52	15.50	54.64	10.56	3.50	54.73	15.23	17.50	54.07	5.86	11.10	49.06	
HippoRAG2	Original	27.52	53.00	61.02	8.90	23.50	50.19	12.32	29.00	51.92	23.99	57.46	58.16	+ 22.40%
	+GraphRAG-R1	38.00	52.00	66.66	20.06	26.50	57.49	32.24	42.50	63.56	35.04	60.00	65.31	

**Figure 3: A comparison of different retrieval formats. Left: the F1 score comparison. Right: the token cost for testing.**

learning and retrieval invocation, which is beneficial for optimizing advanced policies in the later two stages.

Compared to training, which used format, PRA, and CAF rewards simultaneously, we found that the three-stage training strategy delivers superior performance. This approach avoids conflicts or convergence difficulties caused by overly complex reward signals.

4.3.3 Hybrid Graph-Textual Retrieval. To demonstrate the effectiveness of our hybrid graph-textual retrieval design, we compare different retrieval strategies. During training, we adopt three settings: using mixing text and graph (“Text+Graph”), text retrieval only (“Text”), and using graph only (“Graph”). During inference, we utilize 4 different retrieval methods: using 5 text snippets (“Text5”), using 3 text snippets and 10 graph triplets (“Text3+Graph10”), using 1 text snippet and 10 graph triplets (“Text1+Graph10”), and using 20 graph triplets (“Graph20”). The results are shown in Figure 3.

For different training strategies, training only with texts and training only with graph generally lead to performance decrease in the F1 score, demonstrating the effectiveness of mixed training.

For different inference strategies, as the proportion of graph data increases, the number of tokens consumed significantly reduces, while the F1 score of the model only slightly decreases. This demonstrates that our hybrid retrieval approach can achieve a suitable balance between model effectiveness and computational resource consumption.

4.3.4 Integration with Other Retrievers. To answer Q3, we conduct experiments to integrate our proposed method with other existing retrieval methods. As shown in Table 4, GraphRAG-R1 generally improves the performance of different RAG methods across datasets. The last column “Imp” is defined as the average percentage increase in all metrics after integrating GraphRAG-R1 relative to original method. Specifically, GraphRAG-R1 achieves notable average improvements over the original methods, with the highest improvement of 38.37% on LightRAG and consistent gains ranging from 5.26% to 36.25% in other baselines. Note that our method is only trained once and only has access to HippoRAG2 during training, indicating that the reasoning and retrieval ability learned by GraphRAG-R1 can be easily generalized to other GraphRAG methods. More experiments are provided in the Appendix.

5 Conclusion

In this paper, we propose GraphRAG-R1, an adaptive GraphRAG framework to enhance the multi-hop reasoning ability of LLMs through process-constrained RL. Our core designs include the roll-out retrieval enhanced GRPO training method, process-constrained reward designs including PRA, CAF, and phase-depending training, and a hybrid graph-textual retrieval. Experimental results validate the superiority of GraphRAG-R1 over state-of-the-art baselines across both in-domain and out-of-domain datasets. We also demonstrate our method can be flexibly integrated with other retrieval methods. Future works include applying our method to more types of graphs and application scenarios.

Acknowledgments

This work was supported in part by the National Natural Science Foundation of China (No. 92570113, 62472018), the Natural Science Foundation of Tianjin (No. 24JCQNJC02170), and the Tianjin Science and Technology Plan Project (No. 24HHXCSS00001).

References

- [1] Akari Asai, Zeqiu Wu, Yizhong Wang, Avirup Sil, and Hannaneh Hajishirzi. 2024. Self-rag: Learning to retrieve, generate, and critique through self-reflection. (2024).
- [2] Xingyu Chen, Jiahao Xu, Tian Liang, Zhiwei He, Jianhui Pang, Dian Yu, Linfeng Song, Qiuzhi Liu, Mengfei Zhou, Zhuosheng Zhang, et al. 2024. Do not think that much for $2+3=?$ on the overthinking of o1-like llms. *arXiv preprint arXiv:2412.21187* (2024).
- [3] Jhouben Cuesta-Ramirez, Samuel Beaussant, and Mehdi Mounsif. 2025. Large Reasoning Models are not thinking straight: on the unreliability of thinking trajectories. *arXiv preprint arXiv:2507.00711* (2025).
- [4] Huilin Deng, Ding Zou, Rui Ma, Hongchen Luo, Yang Cao, and Yu Kang. 2025. Boosting the generalization and reasoning of vision language models with curriculum reinforcement learning. *arXiv preprint arXiv:2503.07065* (2025).
- [5] Darren Edge, Ha Trinh, Newman Cheng, Joshua Bradley, Alex Chao, Apurva Mody, Steven Truitt, Dasha Metropolitanansky, Robert Osazuwa Ness, and Jonathan Larson. 2024. From local to global: A graph rag approach to query-focused summarization. *arXiv preprint arXiv:2404.16130* (2024).
- [6] Wenqi Fan, Yujuan Ding, Liangbo Ning, Shijie Wang, Hengyun Li, Dawei Yin, Tat-Seng Chua, and Qing Li. 2024. A survey on rag meeting llms: Towards retrieval-augmented large language models. In *Proceedings of the 30th ACM SIGKDD conference on knowledge discovery and data mining*. 6491–6501.
- [7] Leo Gao, John Schulman, and Jacob Hilton. 2023. Scaling laws for reward model overoptimization. In *International Conference on Machine Learning*. PMLR, 10835–10866.
- [8] Yunfan Gao, Yun Xiong, Xinyu Gao, Kangxiang Jia, Jinliu Pan, Yuxi Bi, Yixin Dai, Jiawei Sun, Haofen Wang, and Haofen Wang. 2023. Retrieval-augmented generation for large language models: A survey. *arXiv preprint arXiv:2312.10997* 2, 1 (2023).
- [9] Xinyan Guan, Jiali Zeng, Fandong Meng, Chunlei Xin, Yaojie Lu, Hongyu Lin, Xianpei Han, Le Sun, and Jie Zhou. 2025. DeepRAG: Thinking to Retrieve Step by Step for Large Language Models. *arXiv preprint arXiv:2502.01142* (2025).
- [10] Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shiroong Ma, Peiyi Wang, Xiao Bi, et al. 2025. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948* (2025).
- [11] Zirui Guo, Lianghao Xia, Yanhua Yu, Tu Ao, and Chao Huang. 2024. Lightrag: Simple and fast retrieval-augmented generation. *arXiv preprint arXiv:2410.05779* (2024).
- [12] Bernal Jiménez Gutiérrez, Yiheng Shu, Weijian Qi, Sizhe Zhou, and Yu Su. 2025. From rag to memory: Non-parametric continual learning for large language models. *arXiv preprint arXiv:2502.14802* (2025).
- [13] Haoyu Han, Yu Wang, Harry Shomer, Kai Guo, Jiayuan Ding, Yongjia Lei, Mahantesh Halappanavar, Ryan A Rossi, Subhabrata Mukherjee, Xianfeng Tang, et al. 2024. Retrieval-augmented generation with graphs (graphrag). *arXiv preprint arXiv:2501.00309* (2024).
- [14] Xiaoxin He, Yijun Tian, Yifei Sun, Nitesh Chawla, Thomas Laurent, Yann LeCun, Xavier Bresson, and Bryan Hooi. 2024. G-retriever: Retrieval-augmented generation for textual graph understanding and question answering. *Advances in Neural Information Processing Systems* 37 (2024), 132876–132907.
- [15] Chadi Helwe, Chloé Clavel, and Fabian Suchanek. 2021. Reasoning with transformer-based models: Deep learning, but shallow reasoning. In *2021 International Conference on Automated Knowledge Base Construction (AKBC)*.
- [16] Xanh Ho, Anh-Khoa Duong Nguyen, Saku Sugawara, and Akiko Aizawa. 2020. Constructing A Multi-hop QA Dataset for Comprehensive Evaluation of Reasoning Steps. In *Proceedings of the 28th International Conference on Computational Linguistics*. 6609–6625.
- [17] Edward Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, and Weizhu Chen. 2021. LoRA: Low-Rank Adaptation of Large Language Models. (2021).
- [18] Aaron Jaech, Adam Kalai, Adam Lerer, Adam Richardson, Ahmed El-Kishky, Aiden Low, Alec Helyar, Aleksander Madry, Alex Beutel, Alex Carney, et al. 2024. Openai o1 system card. *arXiv preprint arXiv:2412.16720* (2024).
- [19] Julie Jiang and Emilio Ferrara. 2023. Social-llm: Modeling user behavior at scale using language models and social network data. *arXiv preprint arXiv:2401.00893* (2023).
- [20] Bernal Jimenez Gutierrez, Yiheng Shu, Yu Gu, Michihiro Yasunaga, and Yu Su. 2024. Hipporag: Neurobiologically inspired long-term memory for large language models. *Advances in Neural Information Processing Systems* 37 (2024), 59532–59569.
- [21] Bowen Jin, Hansi Zeng, Zhenrui Yue, Jinsung Yoon, Serkan Arik, Dong Wang, Hamed Zamani, and Jiawei Han. 2025. Search-r1: Training llms to reason and leverage search engines with reinforcement learning. *arXiv preprint arXiv:2503.09516* (2025).
- [22] Dawei Li, Shu Yang, Zhen Tan, Jae Baik, Sukwon Yun, Joseph Lee, Aaron Chacko, Bojian Hou, Duy Duong-Tran, Ying Ding, et al. 2024. DALK: Dynamic Co-Augmentation of LLMs and KG to answer Alzheimer’s Disease Questions with Scientific Literature. In *Findings of the Association for Computational Linguistics: EMNLP 2024*. 2187–2205.
- [23] Shilong Li, Yancheng He, Hangyu Guo, Xingyuan Bu, Ge Bai, Jie Liu, Jiaheng Liu, Xingwei Qu, Yangguang Li, Wanli Ouyang, et al. 2024. GraphReader: Building Graph-based Agent to Enhance Long-Context Abilities of Large Language Models. In *Findings of the Association for Computational Linguistics: EMNLP 2024*. 12758–12786.
- [24] Yuhao Li, Xinni Zhang, Linhao Luo, Heng Chang, Yuxiang Ren, Irwin King, and Jia Li. 2025. G-refer: Graph retrieval-augmented large language model for explainable recommendation. In *Proceedings of the ACM on Web Conference 2025*. 240–251.
- [25] Shengjie Ma, Chengjin Xu, Xuhui Jiang, Muzhi Li, Huiaren Qu, Cehao Yang, Jiaxin Mao, and Jian Guo. 2024. Think-on-graph 2.0: Deep and faithful large language model reasoning with knowledge-guided retrieval augmented generation. *arXiv preprint arXiv:2407.10805* (2024).
- [26] Alex Mallen, Akari Asai, Victor Zhong, Rajarshi Das, Daniel Khashabi, and Hannaneh Hajishirzi. 2023. When Not to Trust Language Models: Investigating Effectiveness of Parametric and Non-Parametric Memories. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 9802–9822.
- [27] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. 2022. Training language models to follow instructions with human feedback. *Advances in neural information processing systems* 35 (2022), 27730–27744.
- [28] Shirui Pan, Linhao Luo, Yufei Wang, Chen Chen, Jiapu Wang, and Xindong Wu. 2024. Unifying large language models and knowledge graphs: A roadmap. *IEEE Transactions on Knowledge and Data Engineering* 36, 7 (2024), 3580–3599.
- [29] Boci Peng, Yun Zhu, Yongchao Liu, Xiaohe Bo, Haizhou Shi, Chuntao Hong, Yan Zhang, and Siliang Tang. 2024. Graph retrieval-augmented generation: A survey. *arXiv preprint arXiv:2408.08921* (2024).
- [30] Fabio Petroni, Aleksandra Piktus, Angela Fan, Patrick Lewis, Majid Yazdani, Nicola De Cao, James Thorne, Yacine Jernite, Vladimir Karpukhin, Jean Maillard, et al. 2021. KILT: a Benchmark for Knowledge Intensive Language Tasks. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*. 2523–2544.
- [31] Parth Sarthi, Salman Abdullah, Aditi Tuli, Shubh Khanna, Anna Goldie, and Christopher D Manning. [n. d.]. Raptor: Recursive abstractive processing for tree-organized retrieval. In *The Twelfth International Conference on Learning Representations*.
- [32] Sheetal S Sonawane and Parag A Kulkarni. 2014. Graph based representation and analysis of text document: A survey of techniques. *International Journal of computer applications* 96, 19 (2014), 1–8.
- [33] Huatong Song, Jinhao Jiang, Yingqian Min, Jie Chen, Zhipeng Chen, Wayne Xin Zhao, Lei Fang, and Ji-Rong Wen. 2025. R1-searcher: Incentivizing the search capability in llms via reinforcement learning. *arXiv preprint arXiv:2503.05592* (2025).
- [34] Yang Sui, Yu-Neng Chuang, Guanchu Wang, Jiamu Zhang, Tianyi Zhang, Jiayi Yuan, Hongyi Liu, Andrew Wen, Shaoshen Zhong, Hanjie Chen, et al. 2025. Stop overthinking: A survey on efficient reasoning for large language models. *arXiv preprint arXiv:2503.16419* (2025).
- [35] Jiahuo Sun, Chengjin Xu, Lumingyuan Tang, Saizhuo Wang, Chen Lin, Yeyun Gong, Lionel Ni, Heung-Yeung Shum, and Jian Guo. [n. d.]. Think-on-Graph: Deep and Responsible Reasoning of Large Language Model on Knowledge Graph. In *The Twelfth International Conference on Learning Representations*.
- [36] Alon Talmor, Jonathan Herzig, Nicholas Lourie, and Jonathan Berant. 2019. CommonsenseQA: A Question Answering Challenge Targeting Commonsense Knowledge. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*. 4149–4158.
- [37] Kimi Team, Angang Du, Bofei Gao, Bowei Xing, Changjiu Jiang, Cheng Chen, Cheng Li, Chenjun Xiao, Chenzhuang Du, Chonghua Liao, et al. 2025. Kimi k1. 5: Scaling reinforcement learning with llms. *arXiv preprint arXiv:2501.12599* (2025).
- [38] Nandan Thakur, Nils Reimers, Andreas Rücklé, Abhishek Srivastava, and Iryna Gurevych. [n. d.]. BEIR: A Heterogeneous Benchmark for Zero-shot Evaluation of Information Retrieval Models. ([n. d.]).
- [39] Harsh Trivedi, Niranjan Balasubramanian, Tushar Khot, and Ashish Sabharwal. 2022. MuSiQue: Multihop Questions via Single-hop Question Composition. *Transactions of the Association for Computational Linguistics* (2022).
- [40] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. *Advances in neural information processing systems* 30 (2017).
- [41] Jingjin Wang. 2025. PropRAG: Guiding Retrieval with Beam Search over Proposition Paths. *arXiv preprint arXiv:2504.18070* (2025).
- [42] Shuohuan Wang, Yu Sun, Yang Xiang, Zhihua Wu, Siyu Ding, Weibao Gong, Shikun Feng, Junyuan Shang, Yanbin Zhao, Chao Pang, et al. 2021. Ernie 3.0 titan: Exploring larger-scale knowledge enhanced pre-training for language understanding and generation. *arXiv preprint arXiv:2112.12731* (2021).

- [43] Yaozu WANG, Qing LI, Zhangjie DAI, and Yue XU. 2024. Current status and trends in large language modeling research. *Chinese Journal of Engineering* 46, 8 (2024), 1411–1425.
- [44] Yu Wang, Nedim Lipka, Ryan A Rossi, Alexa Siu, Ruiyi Zhang, and Tyler Derr. 2024. Knowledge graph prompting for multi-document question answering. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 38, 19206–19214.
- [45] Zhishang Xiang, Chuanjie Wu, Qinggang Zhang, Shengyuan Chen, Zijin Hong, Xiao Huang, and Jinsong Su. 2025. When to use graphs in rag: A comprehensive analysis for graph retrieval-augmented generation. *arXiv preprint arXiv:2506.05690* (2025).
- [46] Tian Xie, Zitian Gao, Qingnan Ren, Haoming Luo, Yuqian Hong, Bryan Dai, Joey Zhou, Kai Qiu, Zhirong Wu, and Chong Luo. 2025. Logic-rl: Unleashing llm reasoning with rule-based reinforcement learning. *arXiv preprint arXiv:2502.14768* (2025).
- [47] Guanming Xiong, Haochen Li, and Wen Zhao. 2025. MCTS-KBQA: Monte Carlo Tree Search for Knowledge Base Question Answering. *arXiv preprint arXiv:2502.13428* (2025).
- [48] Rui Yang, Haoran Liu, Edison Marrese-Taylor, Qingcheng Zeng, Yuhe Ke, Wanxin Li, Lechao Cheng, Qingyu Chen, James Caverlee, Yutaka Matsuo, et al. 2024. KG-Rank: Enhancing Large Language Models for Medical QA with Knowledge Graphs and Ranking Techniques. In *Proceedings of the 23rd Workshop on Biomedical Natural Language Processing*. 155–166.
- [49] Sohee Yang, Elena Gribovskaya, Nora Kassner, Mor Geva, and Sebastian Riedel. 2024. Do Large Language Models Latently Perform Multi-Hop Reasoning?. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 10210–10229.
- [50] Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William W. Cohen, Ruslan Salakhutdinov, and Christopher D. Manning. 2018. HotpotQA: A Dataset for Diverse, Explainable Multi-hop Question Answering. In *Conference on Empirical Methods in Natural Language Processing (EMNLP)*.
- [51] Jingying Zeng, Richard Huang, Waleed Malik, Langxuan Yin, Bojan Babic, Danny Shacham, Xiao Yan, Jaewon Yang, and Qi He. 2024. Large language models for social networks: Applications, challenges, and solutions. *arXiv preprint arXiv:2401.02575* (2024).
- [52] Jingyan Zhang, Dawei Feng, and Bo Ding. 2025. CoRAG: Enhancing Retrieval-Augmented Generation with Coreference Resolution. In *2025 5th International Conference on Artificial Intelligence and Industrial Technology Applications (AIITA)*. IEEE, 2116–2123.
- [53] Qinggang Zhang, Shengyuan Chen, Yuanchen Bei, Zheng Yuan, Huachi Zhou, Zijin Hong, Junnan Dong, Hao Chen, Yi Chang, and Xiao Huang. 2025. A survey of graph retrieval-augmented generation for customized large language models. *arXiv preprint arXiv:2501.13958* (2025).
- [54] Kishi Zhu, Xiaoming Guo, Shengting Cao, Shenglin Li, and Jiaqi Gong. 2024. StructuGraphRAG: Structured document-informed knowledge graphs for retrieval-augmented generation. In *Proceedings of the AAAI Symposium Series*, Vol. 4. 242–251.

A More Experiments

A.1 Experiments with Qwen2.5-7B-Instruct

In this study, we further explore the performance of GraphRAG-R1 on instruct-tuned models. We observed that results based on the original Qwen2.5-7B-base still have room for improvement on certain tasks, particularly in terms of instruction compliance and contextual understanding. Therefore, we selected the Qwen2.5-7B-Instruct model as the backbone. Compared to the base model, the instruct-tuned model demonstrates stronger capabilities in understanding user intent, following output formats, and handling multi-turn dialogues. Based on this model, we retained the original GraphRAG-R1 framework and only performed adaptive tuning on the training strategy, prompt templates, and inference hyperparameters, aiming to verify the stability and scalability of our method across different types of backbone models.

The experimental results demonstrate that the tuned GraphRAG-R1 with Qwen2.5-7B-Instruct not only significantly outperforms the base version but also achieves the best performance on most evaluation metrics across datasets such as HotpotQA, MuSiQue, 2Wiki, and PopQA. Detailed performance comparisons are provided in Table 5.

Furthermore, we tested the integration of our approach with other retrievers, as shown in Table 6. In all tested configurations, GraphRAG-R1 consistently improved performance over the original retrieval methods, demonstrating strong transferability and generalization. This indicates that our framework can be flexibly adapted to different retrieval mechanisms.

A.2 Experiments with LLaMA-3-8B

In this subsection, we investigate the effectiveness of GraphRAG-R1 across different base models. We adopt another representative open-sourced LLM, Llama-3-8B, as the backbone LLM. The experimental setting is similar to using Qwen2.5-7B, except we add PRA reward in the third training stage to encourage the model to invoke retrieval, which enhances performance. The overall results are presented in Table 7. Collectively, these experiments demonstrate that our method is effective and outperforms baselines when using LLaMA-3-8B as the backbone LLM, showing that our method is generalizable across different LLM models.

A.3 Hyperparameter Sensitivity Analysis

To evaluate the hyperparameter robustness of the GraphRAG-R1 framework, we conducted sensitivity analysis on four key parameters. The results are shown in Figure 4 and demonstrate that the performance of GraphRAG-R1 has a wide robustness interval around the parameters selected in the main experiment, which enhances the reproducibility and practical reliability of the method.

A.4 Cost and Efficiency Analysis

In this subsection, we evaluate the performance and cost of different reasoning-enhanced retrieval methods. We compare our approach with KGP and ToG, two reasoning-enhanced retrieval methods that also support multiple retrievals. In addition to comparing the performance measured by F1 scores, we also calculate the token consumption throughout the retrieval and reasoning processes. Specifically, for ToG and KGP, we count the number of accesses to the knowledge graph, for our method, we tally the frequency of calls to external retrieval tools. Token overhead accounts for consumption throughout the retrieval and reasoning processes. As shown in Table 8, GraphRAG-R1 achieves significant improvements in both performance and computational cost by deeply integrating the intrinsic reasoning capabilities of LLMs. Compared to baseline methods that also leverage LLM reasoning, our method demonstrates substantially improved answer quality while requiring lower token overhead than the other two methods. This validates the effectiveness and cost efficiency of our approach.

A.5 Cross-Domain Adaptation Verification

GraphRAG-R1 is designed as a general framework agnostic to specific graph structures, tasks, and domains. To verify its domain generalization and compatibility, we adapted an advanced retrieval method DALK [22] originally designed for the medical domain, integrated it into our framework, and evaluated it on general-domain datasets. We only replacing GraphRAG-R1’s underlying retrieval tool with the adapted DALK retriever. As shown in Table 9, the integrated framework achieved a significant improvement compared to the original DALK method, with a relative F1 score increase of

Table 5: The results of different methods for multi-hop question answering using Qwen2.5-7B-Instruct. The first three datasets are in-domain, and PopQA is out-of-domain (i.e., unseen during training). The best results are in bold and the second-best results are underlined.

Method	HotpotQA			MuSiQue			2Wiki			PopQA		
	F1	ACC _L	SBERT	F1	ACC _L	SBERT	F1	ACC _L	SBERT	F1	ACC _L	SBERT
Naive RAG	73.44	<u>79.50</u>	86.41	31.61	33.00	65.67	41.53	42.00	69.69	<u>51.54</u>	<u>62.90</u>	73.92
KGP	45.90	51.00	70.56	10.09	13.00	50.04	17.73	22.50	55.56	42.39	54.50	69.21
ToG	20.41	25.00	56.73	10.30	12.00	49.66	29.13	29.50	62.16	30.67	35.00	63.46
LightRAG	42.74	47.00	70.84	18.92	16.50	57.84	29.59	30.00	62.72	47.65	59.00	71.73
PropRAG	71.11	77.50	85.71	38.88	<u>42.00</u>	<u>69.40</u>	<u>60.31</u>	<u>63.00</u>	<u>79.45</u>	44.43	58.30	69.16
G-retriever	15.74	19.50	55.52	6.19	6.00	48.49	12.80	52.20	12.00	2.52	4.10	43.82
HippoRAG2	<u>73.91</u>	78.00	<u>86.44</u>	<u>40.16</u>	39.00	69.27	51.28	54.00	75.63	51.27	63.00	<u>74.05</u>
GraphRAG-R1	74.21	80.00	87.40	49.63	50.00	74.27	68.28	74.00	83.57	57.91	61.50	79.75

Table 6: The results of integrating GraphRAG-R1 using Qwen2.5-7B-Instruct with other GraphRAG retrieval methods. Imp denotes the average improvement across all datasets.

Method	Model	HotpotQA			MuSiQue			2Wiki			PopQA			Imp
		F1	ACC _L	SBERT	F1	ACC _L	SBERT	F1	ACC _L	SBERT	F1	ACC _L	SBERT	
Naive RAG	Original	73.44	79.50	86.41	31.61	33.00	65.67	41.53	42.00	69.69	51.54	62.90	73.92	+10.79%
	+GraphRAG-R1	71.51	80.00	85.30	51.96	53.50	75.38	56.77	67.00	76.06	42.29	59.60	68.55	
KGP	Original	45.90	51.00	70.56	10.09	13.00	50.04	17.73	22.50	55.56	42.39	54.50	69.21	+24.35%
	+GraphRAG-R1	52.27	60.50	75.66	27.26	27.50	62.76	44.83	55.00	70.92	33.99	49.70	64.46	
ToG	Original	20.41	25.00	56.73	10.30	12.00	49.66	29.13	29.50	62.16	30.67	35.00	63.46	+19.85%
	+GraphRAG-R1	21.44	27.00	61.63	21.99	21.50	59.28	44.96	54.50	70.61	27.24	37.20	60.83	
LightRAG	Original	42.74	47.00	70.84	18.92	16.50	57.84	29.59	30.00	62.72	47.65	59.00	71.73	+27.99%
	+GraphRAG-R1	56.12	63.00	77.40	40.10	40.00	68.21	55.99	63.40	76.13	42.74	57.10	69.57	
PropRAG	Original	71.11	77.50	85.71	38.88	42.00	69.40	60.31	63.00	79.45	44.43	58.30	69.16	+6.86%
	+GraphRAG-R1	69.81	78.00	83.44	49.94	52.50	74.97	68.23	77.50	82.51	44.22	60.30	69.93	
G-retriever	Original	15.74	19.50	55.52	6.19	6.00	48.49	12.80	52.20	12.00	2.52	4.10	43.82	+45.22%
	+GraphRAG-R1	31.47	30.50	66.45	16.13	12.00	56.87	22.96	22.50	59.56	15.74	16.90	53.92	
HippoRAG2	Original	73.91	78.00	86.44	40.16	39.00	69.27	51.28	54.00	75.63	51.27	63.00	74.05	+9.87%
	+GraphRAG-R1	74.21	80.00	87.40	49.63	50.00	74.27	68.28	74.00	83.57	57.91	61.50	79.75	

Table 7: The results of different methods for multi-hop question answering using Llama-3-8B as the backbone LLM. The best results are in bold and the second-best results are underlined.

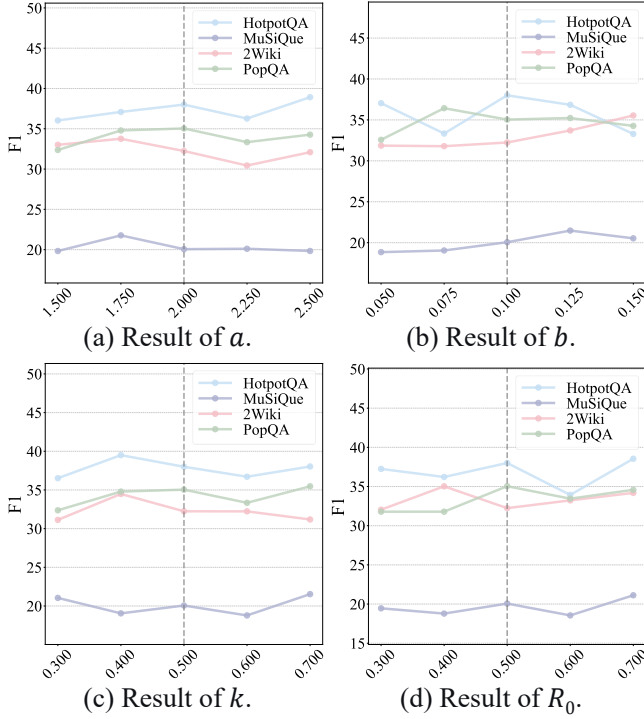
Method	HotpotQA			MuSiQue			2Wiki			PopQA		
	F1	ACC _L	SBERT	F1	ACC _L	SBERT	F1	ACC _L	SBERT	F1	ACC _L	SBERT
Naive RAG	9.57	27.00	48.25	1.17	4.00	40.11	4.14	10.00	43.58	8.14	24.70	46.59
KGP	3.61	17.00	46.41	1.38	3.50	44.49	<u>8.23</u>	26.50	<u>48.48</u>	10.75	59.00	48.13
ToG	8.97	16.50	<u>49.83</u>	3.28	10.00	<u>44.74</u>	7.91	23.00	48.36	<u>10.96</u>	37.50	<u>49.57</u>
LightRAG	5.33	12.50	43.95	1.83	3.50	41.51	4.68	12.00	43.95	6.71	27.50	45.83
PropRAG	<u>12.47</u>	23.00	49.03	<u>3.83</u>	6.50	42.17	5.06	11.50	43.06	9.76	25.70	48.10
G-retriever	1.91	23.00	43.09	1.06	<u>12.50</u>	42.77	2.43	18.00	42.26	0.25	38.10	39.32
HippoRAG2	6.89	22.00	46.06	1.85	4.00	42.03	5.61	12.00	43.51	7.85	26.10	46.57
GraphRAG-R1	13.16	<u>26.50</u>	53.06	6.32	16.00	48.96	13.36	<u>25.00</u>	51.46	11.64	<u>41.50</u>	50.42

Table 8: Performance and cost comparison with KGP and ToG, two reasoning-enhanced GraphRAG methods

Method	HotpotQA		MuSiQue		2Wiki		PopQA	
	F1	#Token	F1	#Token	F1	#Token	F1	#Token
KGP	10.73	2687.44	4.61	2858.23	10.16	2173.02	21.01	3182.02
ToG	11.44	3136.61	5.02	2873.56	14.45	2065.94	29.21	2838.49
GraphRAG-R1	38.00	1674.47	20.06	2086.10	32.24	1883.21	35.04	1449.80

Table 9: The results of integrating GraphRAG-R1 with other GraphRAG retrieval methods. Note that GraphRAG-R1 is only trained once and do not access other RAG methods during training. Imp denotes the average improvement across all datasets.

Method	Model	HotpotQA			MuSiQue			2Wiki			PopQA			Imp
		F1	ACC _L	SBERT	F1	ACC _L	SBERT	F1	ACC _L	SBERT	F1	ACC _L	SBERT	
Dalk	Original	0.62	7.50	41.96	0.40	3.00	40.64	0.75	4.00	41.66	0.14	8.80	39.37	+ 69.50%
	+GraphRAG-R1	15.95	20.50	52.38	8.15	8.50	48.33	10.45	14.40	49.54	13.38	26.60	51.90	

**Figure 4: Sensitivity analysis of key hyperparameters in GraphRAG-R1. Subfigures (a)-(d) illustrate the variation in F1 score on the four validation sets as the key hyperparameters are varied within specified ranges. Other hyperparameters were fixed at their main experiment values. The gray dashed lines indicate the parameter values selected for the main experiments.**

approximately 69.50% on Qwen2.5-7B. This strongly proves that our framework can flexibly compatibly integrate and enhance specialized retrievers from different domains.

Table 10: Dataset Statistics

Metric		MuSiQue	2Wiki	HotpotQA	PopQA
Graph	# Passages	11,656	6,119	9,221	8,676
	# Nodes	91,729	42,694	82,157	74,472
	# Triplets	107,448	50,671	98,709	139,273
Questions	Total	1,000	1,000	1,000	1,000
	Training Set	800	800	800	0
	Testing Set	200	200	200	1,000

A.6 More Details about Datasets

We adopt four widely used benchmark datasets, including HotpotQA, MuSiQue, and 2Wiki from HippoRAG [20]¹ and one simple QA dataset PopQA from PropRAG [41]². Detailed descriptions of the datasets are as follows.

- **HotpotQA** [50]: A Wikipedia-based dataset where answering each question requires reasoning over multiple supporting documents.
- **MuSiQue** [39]: A challenging dataset consisting of questions composed of 2 to 4 single-hop questions, requiring multi-step reasoning.
- **2Wiki** [16]: Another Wikipedia-based dataset where each question requires information from two articles to be correctly answered.
- **PopQA** [26]: A dataset of entity-centric QA pairs, where each question is generated by converting a knowledge tuple retrieved from Wikidata.

The statistics of these datasets are shown in Table 10.

¹<https://github.com/OSU-NLP-Group/HippoRAG/releases/tag/v1.0.0>

²<https://github.com/ReLink-Inc/PropRAG/tree/main/outputs/popqa>